- Having too many idle connections
- *Transaction ID wraparound*
- Runaway queries, or zombie queries, that never return results to the client

Some of the safeguards you'll add are:

- Setting a statement_timeout to set a max allowed amount of time for a statement to run
- Setting a lock_timeout to prevent waiting too long to acquire a lock
- Setting idle_in_transaction_session_timeout[13] to protect against transactions that are idle and not performing any work

You've already seen statement_timeout and lock_timeout in an earlier chapter.

Focus now on some of the idle timeouts. The following table presents these parameters and some recommended values in milliseconds.

Parameter Name	Default	Tuned Value	Restart
idle_session_timeout (14+)	0	30000	false
idle_in_transaction_session_timeout	0	3000	false
statement_timeout	0	10000	false
lock_timeout	0	3000	false

Table 5—Database Concurrency Parameters

Tuning parameter values

When considering idle_session_timeout[14] and idle_in_transaction_session_timeout, start by adjusting the latter.

Recommended parameter values are from the *Tightly Coupled* post *GOTO Postgres Configuration for RDS and Self-managed Postgres*[15] and *My GOTO Postgres Configuration for Web Services.*[16].

To collect the best values for your particular database, conduct experiments and gradually introduce changes in small increments. Develop a good understanding of how you'll observe the effect and make sure it provides the benefit you're looking for.

Note that Cloud providers will tune some parameters like memory allocation dynamically based on the server instance. Client parameters like timeouts may be left up to customers to enable and adjust.

13. https://www.postgresql.org/docs/current/runtime-config-client.html
14. https://postgresqlco.nf/doc/en/param/idle_session_timeout/
15. https://tightlycoupled.io/goto-postgres-configuration-for-rds-and-self-managed-postgres/
16. https://tightlycoupled.io/my-goto-postgres-configuration-for-web-services/